

Yield Not Thy Core

Achilles Benetopoulos
UC Santa Cruz, USA
abenetop@ucsc.edu

Andi Quinn
UC Santa Cruz, USA
aquinn1@ucsc.edu

Robert Soulé
Yale University, USA
robert.soule@yale.edu

Peter Alvaro
UC Santa Cruz, USA
palvaro@ucsc.edu

Abstract

Programming distributed systems has never been easier, but making them perform well and efficiently exploit available resources remains difficult. Fast programs on a single host slow to a crawl when given more resources, while systems designed for scale squander resources on modest problems.

We present Magpie, a programming model and framework for building high-performance, data-intensive applications. Magpie provides the straightforward guarantee that fast code remains fast when distributed. Because application code performs best when executed without interruptions, Magpie takes protocol off the critical path, automatically orchestrating the placement of data and computation to ensure uninterrupted local compute whenever possible. This flexibility requires extreme *adaptivity*, postponing all decisions about movement and placement until runtime. To support this fine-grained adaptivity, distributed computations in Magpie are structured as compositions of lightweight transactions over distributed objects, both of which may move freely through the system.

To showcase Magpie's performance and generality, we develop two very different libraries: a high-performance key-value store and a graph processing system. We show that Magpie achieves parity with or outperforms state of the art systems in both areas while providing richer guarantees.

CCS Concepts: • **Computer systems organization** → **Cloud computing**; *Client-server architectures*; • **Computing methodologies** → **Distributed computing methodologies**.

ACM Reference Format:

Achilles Benetopoulos, Andi Quinn, Robert Soulé, and Peter Alvaro. 2026. Yield Not Thy Core. In *21st European Conference on Computer Systems (EUROSYS '26)*, April 27–30, 2026, Edinburgh, Scotland Uk. ACM, New York, NY, USA, 19 pages. <https://doi.org/10.1145/3767295.3803616>

1 Introduction

Much of distributed systems research over the past few decades has focused on questions of scale, i.e., how we can

do more with more [42, 44]. This is driven by increasingly large data sets and clusters. As a consequence of this focus, it has become easier than ever to write distributed programs; by using any of a myriad of frameworks, one can quickly spin up a job that runs on thousands of machines with a few dozen lines of code.

Lost in this focus on scale, however, has been an examination of efficiency, i.e., how to do more with less [29]. As there is increased pressure from cost (e.g., expenses in running a datacenter) and environmental impact (e.g., how much power do my machines consume), it will be increasingly important to get the most out of a limited set of machines.

In striving for efficiency, our goal is to keep CPUs on a machine productively busy, avoiding whenever possible the need to pause for coordination (e.g. to provide consistency guarantees) or I/O (e.g. the dreaded shuffle step). Achieving this goal requires addressing one of the classic unsolved issues in distributed systems: *the placement problem*.

Solving the placement problem requires deciding where (i.e., on which hosts) an application's data should reside and where its computation should occur; a poor solution to the placement problem can be catastrophic for performance [22]. For example, if the application places frequently co-accessed data on separate hosts, it will engage in distributed coordination on its hot path. Regardless of the form this coordination takes (e.g., distributed locking, distributed atomic commit or consensus protocols, or even simple acknowledgment messages) it invariably interrupts what would otherwise be compute-bound local computation.

Existing programming models do not adequately address the placement problem. At one extreme, message-passing models (including distributed actors [12, 19] and RPC-based service-oriented architectures [26, 36, 38]) require the developer to address the placement problem at design time, resulting in rigid systems whose overheads increase as access patterns change. At the other extreme are systems that abstract away communication entirely by presenting a global view of data, such as distributed shared memory [6, 24] and distributed data analytics frameworks [13, 27, 42, 44]. Such systems solve the placement problem on-demand, making them (at least in principle) more adaptive to changing workloads and better able to spread load across existing resources. Unfortunately, by completely hiding the movement (and hence interruption) of computation over data, they make it impossible for programmers to effectively overlap communication with local work. As diverse as the existing models may be, they share a common shortcoming: they squander



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

EUROSYS '26, Edinburgh, Scotland Uk

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2212-7/2026/04

<https://doi.org/10.1145/3767295.3803616>

resources by forcing what would otherwise be fast, local computation to stall.

In this paper, we present Magpie, a programming model and distributed runtime for implementing efficient distributed applications and infrastructure. Magpie finds a middle ground between systems that provide abstractions for communication and those that mask it. It exposes a programming model that enables programmers to dictate when local computation must proceed uninterrupted and when it must wait for data dependencies to be satisfied, allowing the infrastructure to delay all compute and data placement choices until the last possible moment. Magpie bundles memory and storage into bounded, movable objects, and computation into bounded, movable, non-preemptible nanotransactions. Nanotransactions expose a simple, performance-oriented contract based on the principle of uninterrupted local computation: all of a nanotransaction’s operations run without interruption on a single host, where all of its data dependencies (objects) are already located. Magpie places a nanotransaction and its required data on the same host before executing the nanotransaction, making such decisions at runtime. The system will move the nanotransaction, its data, or both as necessary; but, ideally, and in the common case, will move neither.

Using Rust and Linux, we implemented a prototype of the Magpie runtime running on commodity hardware, and built two reusable libraries to showcase the system’s generality, performance, and parsimonious use of resources. To highlight how the overheads introduced by Magpie are balanced by the benefits of uninterrupted computation even in latency-critical datacenter infrastructure, we implement a distributed key-value store which we show to be competitive with the state of the art (i.e., Redis and Memcached) while providing stronger guarantees, including durable and atomic writes. We then show how Magpie’s adaptivity allows it to improve throughput (by a factor of 1.6 and 2, respectively) by changing the placement of computation in response to more complex operations. To showcase the benefits of adaptive placement and unyielding compute, we construct a distributed graph analytics library, and demonstrate how Magpie’s core principle enables it to accelerate long-running tasks that exhibit strong temporal locality, outperforming GraphX by 1.42-5.77x on PageRank workloads.

2 Motivation

In this section, we motivate the need for adaptive compute and data placement by demonstrating the performance consequences of early placement decisions, presented via two examples chosen for their simplicity and generality.

2.1 Compute should be mobile and late-bound

Consider an application that processes large amounts of data stored on a server, like a depth-first search of a graph, modeled using adjacency lists; Fig. 1 shows the running time

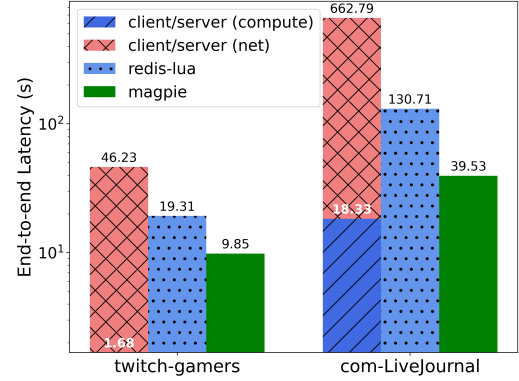


Fig. 1. Latency of depth-first graph traversal (log scale): The twitch-gamers graph has ~168k vertices and ~6.8M edges, while the com-LiveJournal graph has ~4M vertices and ~35M edges.

of this search on two different undirected graphs from the SNAP library [3], under three different system configurations, which are described below. We report the average time of five experimental runs; RTTs between hosts were between 90-110 μ s.

The first configuration (client/server) consists of a client Rust program that maintains the search state and drives the exploration, fetching unexplored vertices’ adjacency lists from a database (in this case, Redis). Fast, local code on a single machine (client/server (compute)) frequently stalls waiting for the network (client/server (net)), with detrimental effects to performance; the costs of network interactions dominate the execution time, accounting for 96-97% of the total running time.

The second configuration (redis-lua) leverages Redis’ support for User Defined Functions (UDFs) written in the Lua programming language. Despite using a slower interpreted language, this variant exhibits between 2.4-5.1x lower latency than the client/server variant, thanks to the benefits of near-memory computation.

Both of the above configurations are stand-ins for typical message-passing system designs. The first configuration represents architectures where components like stateless services perform local processing on data that has been retrieved from a data store, whereas the second one represents cases where some client performs function shipping to a stateful actor which implements functions of arbitrary complexity over local data in the form of message handlers.

A fundamental limitation of this dichotomy between data fetching versus function shipping is that it presents users with a binary choice to the placement problem: either the target data needs to be *copied* from the host that stores it to the host that will process it or the function to be executed will be sent to the target host. Programmers are responsible for choosing correctly at design time, while trying to

satisfy a number of competing constraints: individual computations need to complete quickly, suggesting that function shipping would always be preferable, but at the same time load needs to be spread evenly across a cluster, which would be impossible in cases where hot data is stored on a single host. Furthermore, while function shipping and computation pushdown can be applied with relative success to computations like database queries that exhibit constrained access patterns [43, 45], arbitrary programs exhibit data-dependent control-flow, necessitating the involvement of the client and negating the benefits of function shipping in terms of network latency [30], as well as concurrent reads and updates of shared data, requiring mechanisms for mutual exclusion between the actions of different clients, potentially exacerbating network traffic.

The third configuration in Fig. 1 leverages Magpie’s abstractions (§3) to achieve the same near-memory computation, proceeding entirely independently of the client and achieving a speedup of 1.96–3.3x over the UDF-based approach. In this case, Magpie behaves like the function shipping variant, keeping the graph data on the server it was originally hosted on, but as we discuss later, this was not a choice the user had to make. The same program that shipped the search computation to the graph data could have also caused the data to move to a host with different compute capacity or even, if it exhibited latent parallelism to exploit (§3.1.4), spill over many hosts.

2.2 Data should be mobile and late-bound

In some scenarios the choice that maximizes performance is neither to copy data temporarily to a host for processing nor to ship a computation to the place where the data resides. As workloads and data distributions change over time, placement decisions that performed well under one regime can start to compromise application performance, and their efficiency can be fleeting [10].

We set up an experiment that illustrates how design-time assumptions can break down under shifting workloads, and cause considerable performance penalties for applications. We issue multi-get operations for two keys from a closed-loop client for a period of 10s against two key/value stores, Memcached and the KVS Magpie library we describe in §4. Memcached performs client-side partitioning of keys to available servers, while Magpie distributes keys over a set of storage objects using a form of rendezvous hashing. Despite being focused on key/value stores, this experiment characterizes the behavior of any system that performs statistical partitioning of data, such as data analytics frameworks (e.g. Spark, MapReduce), where the eventual data placement is decoupled from its concrete access patterns. The results are presented as a timeseries of client-side latency in Fig. 2.

At the start of the experiment data is partitioned randomly, and so Magpie has to delay executing the get operations until it has co-located the storage of the target keys. For the

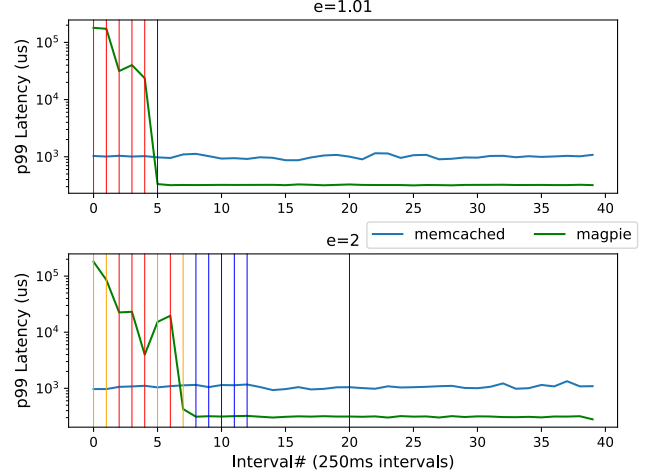


Fig. 2. p99 latency of 2-key get (log scale): Both systems are running on four hosts. Keys are selected following a Zipfian distribution with an exponent value of 1.01 (top) and 2 (bottom). Vertical bars illustrate the number of data moves in each interval; blue means at most 1 movement, orange means between 2 and 5, and red is anything higher than that.

Zipfian distribution with a value of $e = 1.01$, this initial phase approximates a mostly uniform sampling of the underlying keyspace, under which a static partitioning that evenly balances keys over a set of available servers is a winning strategy. In contrast, Magpie’s emphasis on maximizing locality leads to higher tail latencies as a large number of storage buckets is moved to the host servicing the client’s requests—while Magpie reconfigures the placement of storage buckets between hosts, Memcached’s tail latencies are up to two orders of magnitude lower.

As the workload evolves however, and more of the buckets holding the pairs of co-accessed keys have already been consolidated on the same server, the benefits of adaptive data placement start to become noticeable. Within a timespan of one to two seconds, Magpie reaches a steady state in which the target server can start responding to client queries without further network interactions, thereby quickly eclipsing the performance of the baseline. The benefits of consolidating data dependencies becomes more pronounced when the workload becomes more skewed (for $e = 2$), as avoiding repeated network calls across multiple locations amortizes the cost of relatively expensive data movement.

This experiment demonstrates the importance of providing the infrastructure with the freedom to react to application access patterns and to adapt data and compute placement to minimize interruptions on an application’s critical path, especially when applications exhibit windows of temporal locality, frequently the case in large-scale data processing (§5.3).

2.3 Goals

Despite their simplicity, these examples illustrate how any solution to the placement problem that commits *early* is fragile. Static partitioning strategies adversely impact system throughput when workloads don't match their assumptions, and place an additional burden on programmers during application design. Furthermore, placement rigidity can lead to pervasive distributed coordination, which is detrimental to performance and is prohibitively expensive for applications with strict latency requirements.

In section §2.1 we demonstrated that in otherwise short-running computations, the overheads of distributed protocol dominate their running time and are detrimental to program performance. Therefore, we aim to eliminate coordination from the application's critical path, and let user programs **execute uninterrupted** for as long as possible. This allows us to support programs with μ s-scale latency requirements.

Providing user programs with uninterrupted compute requires **extreme adaptivity** in the underlying system's placement of data and computation, since changes in workloads over time prevent programmers from establishing "optimal" data partitioning schemes [4, 20]. This emphasis on extreme adaptivity implies that user programs should make no reference to *where* data may be located, or where code should execute—after all, if we want systems to be able to dynamically and freely adjust placement of compute and data, user programs should not need to take any special actions based on locations (i.e. hosts). These actions include direct communication—in our system user programs do not exchange messages, and communication happens "outside" of user programs in response to their execution.

Our ultimate goal is to provide application programmers with a minimal but broadly reusable set of abstractions that enable us to address the most common and pernicious distributed systems concerns entirely at the infrastructure level. In addition to the automated data and compute placement that maximizes efficiency and resource utilization, we want to help users with the coordination of distributed action in the presence of partial failure, by providing low-cost concurrency control for safe access to shared data.

Instead of providing users with low-level mechanisms (e.g. distributed locks and leases) that they need to leverage to enforce correctness of concurrent, distributed operations, we aim to provide **top-down data consistency guarantees** similar to a database, in which the chosen consistency model determines the permissible concurrency level, but also how to automatically roll back effects of aborted operations. Conventional wisdom dictates that automatic enforcement of database-level guarantees for arbitrary distributed computation is too costly—in this paper we show this to be false.

3 Design

We begin this section by showing how the goals from the previous section are embodied in Magpie's design, based on portable units of globally-addressable data and fluid computation. We describe how this design enables Magpie to provide strong correctness guarantees automatically and at low cost.

3.1 Programming Model

Magpie exposes two core abstractions: data is organized in **objects**, and computation is expressed in the form of shippable functions called **nanotransactions**. Objects are distributed according to a notion of **object ownership**.

3.1.1 Objects. At its core, an object is a versioned, typed region of memory for the allocation of semantically related structured data [18]. Each object possesses a unique identity in a global address space, similar to Twizzler [9]. In order to minimize the costs of last-minute data movement decisions, any references, either intra or inter-object, are expressed in the form of **invariant pointers** (tuples of object IDs and offsets within the target objects) independent of any ephemeral context like a process address space, making them directly byte-portable.

Programmers will, in general, not use standard library data structures when populating objects, for two reasons: references should be invariant in order to be compatible with our execution model, and the data structures should make pre- and post-images of updates available to the system to facilitate recovery, in service of our top-down consistency goal. We provide object-aware, persistent collection types to replace standard library collection types with these features.

Since programmers allocate pieces of data that are expected to be co-accessed into the same object, objects represent the unit of locality for our runtime. Whenever the runtime moves data between hosts, it moves entire objects, and in this way we avoid placing co-accessed data on different hosts (which could happen if we, for instance, followed a hash-based data partitioning strategy), allowing us to provide our uninterrupted execution guarantee at access time.

3.1.2 Nanotransactions. User programs access and modify objects through a mechanism called nanotransactions. Like database transactions, nanotransactions capture user-supplied application logic and read and modify a collection of objects under ACID guarantees.

The "nano" prefix of nanotransactions does not necessarily refer to the nanotransactions' expected execution time — since nanotransactions are non-preemptible, their execution time is only bounded by the amount of data they process. Rather, it refers to the amount of mechanism needed to support their execution. Nanotransactions are *never distributed*. It is the runtime's job to co-locate a nanotransaction with its data dependencies on whichever host they can be met,

orchestrating data movement if no such location exists. This just-in-time action is enabled by the fact that, at submission time, we know all the nanotransaction’s data dependencies, in the form of the invariant pointers passed to it as arguments. If a nanotransaction’s dependencies can be satisfied on a single host, then transaction execution can proceed entirely locally, and it is guaranteed to not interfere with any other nanotransactions’ execution except for those with intersecting data dependencies.

Having placed a nanotransaction’s dependencies on a single host before beginning execution, we can provide the guarantee that it will run to completion without engaging in data movement; without the object abstraction, our runtime would have no way of knowing the extent of a nanotransaction’s data accesses, and we would therefore have to rely on low-level signals like page faults, and react to them during a nanotransaction’s execution. Additionally, executing user programs transactionally allows programmers to focus their error handling only on application-level errors, and ignore infrastructure and coordination-level errors.

The process of co-locating code with its data dependencies is driven by a system of ownership, which we discuss further in the following section.

3.1.3 Ownership. Orchestration of the movement of data and code is built on top of an ownership model [41], in which at any given point in time each object in the cluster is owned by exactly one host. An object’s owner is the only host that may execute transactions that modify the given object, enforcing a single-writer principle which eliminates the need for a distributed commit protocol.

The activation of a nanotransaction on the host that exclusively owns its data dependencies has a direct effect on the performance of the transaction system implementation. Ownership means that we can freely query and modify data without the need to perform any protocol, which means that providing *serializable* transaction semantics as a default is cheap. This is a critical property in our system, because serializability represents the expected behavior of a non-concurrent, non-distributed system for programmers.

Ownership, together with the location independence of nanotransactions and objects facilitate the adaptivity of our system, enabling us to take an extreme late-binding approach to the relative placement of data and compute in the cluster, postponing decisions until the moment a nanotransaction’s execution is requested.

3.1.4 Epics. The same two features that enable both the performance and strong guarantees of nanotransactions — knowing the entirety of their data dependencies ahead of time, and being able to execute to completion in one place — also limit their expressivity. Combined, these features mean we cannot express computations that may, for instance, fan out to multiple objects across hosts, such as a traversal of a composite data structure through pointer chasing, using

just nanotransactions. Additionally, nanotransactions don’t allow programmers to take advantage of parallelism, as they execute in a single thread (§3.2.1).

To overcome these restrictions, we introduce a workflow abstraction for the composition of nanotransactions, which we refer to as an **epic**. Epics are a dynamic task graph abstraction [33], capturing computations that span multiple objects that are not known ahead of time, as well as computations with parallelizable subcomputations. Vertices in the epic task graph correspond to nanotransactions, and edges between tasks correspond either to a “spawns” relationship or to control and data dependencies between tasks. These edges also represent the points where the runtime will need to interpose to perform functions such as object discovery or result propagation.

Spawning a nanotransaction is analogous to instantiating a future that is not considered a candidate for scheduling until its spawner commits. In each set of spawned nanotransactions, an edge (u, v) between tasks u and v that corresponds to a control dependency means that the downstream task v needs to wait for the completion of the upstream task u before executing; a data dependency edge subsumes a control dependency, but additionally means that task u will output a result that will be passed as an argument to task v . Tasks with no incoming dependency edges are considered immediately schedulable as well as independent of other tasks, and so will be scheduled in parallel to each other, subject to potential overlaps in their object dependencies and availability of resources. Our approach of explicit “articulation” of a program’s execution by the programmer allows us to extract coordination overheads out of an application’s critical path and simplify fault handling, while permitting the programmer to provide guidance on independent computations that can run in parallel.

Like sagas [17], epics are fundamentally coordinations of *independently* atomic actions (nanotransactions) that achieve an atomic global outcome, in the sense that all or none of their effects become visible. Unlike sagas, epics also provide isolation from the effects of concurrent operations (i.e., other nanotransactions/epics), but unlike simple nanotransactions, epics do not provide *serializable* isolation by default. Epics by default execute under Snapshot Isolation [7], by establishing a lazily populated snapshot over their read set and deferring commit of modifications until the entire epic has been executed to completion. If a conflict is detected, the epic aborts and discards its local changes; otherwise it commits and installs them.

3.2 System Components

A Magpie cluster consists of two kinds of components: workers (§3.2.1), which maintain ownership of data and execute nanotransactions, and a global ownership orchestrator (§3.2.2) that caches ownership state for objects in the cluster and is responsible for triggering data ownership changes

between workers in response to workload and capacity fluctuations. Workers also maintain an object ownership cache, used to identify the owning worker of known objects without contacting the global ownership orchestrator.

3.2.1 Workers. Magpie workers are responsible for executing user transactions. Fig. 3 shows a high-level breakdown of a worker’s transaction subsystem, which will serve as a reference during discussion of a worker’s function.

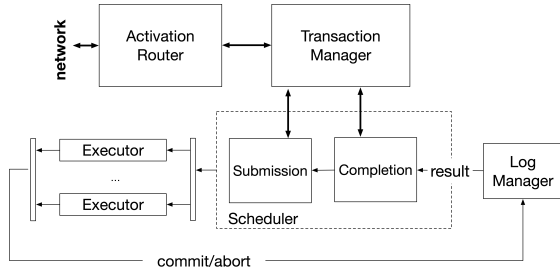


Fig. 3. Transaction Execution Subsystem of a Magpie worker

To execute a nanotransaction, a worker must be able to satisfy its data dependencies locally. If a nanotransaction can be executed locally, it is submitted to the worker’s local nanotransaction scheduler.

Each worker is responsible for the scheduling of local nanotransactions. Similar to other systems [23, 25], we follow an affinity-based scheduling strategy that extends the ownership model down to a worker’s executor pool so that only one executor thread has ownership of a given object (and is therefore responsible for executing mutating transactions on it) at any given time. This single-writer design means that executor threads do not need to acquire locks on objects in order to access them. Nanotransactions whose data dependencies overlap will end up forming a total order; this means that we can eliminate synchronization mechanisms from user programs.

If a worker cannot execute a nanotransaction locally due to missing data dependencies, it will consult its local ownership cache to identify a remote worker where they can be met. Failing that, it enlists the help of the global ownership orchestrator, whose role we discuss next.

3.2.2 Ownership Orchestrator. The global ownership orchestrator maintains a mapping of object IDs to owning workers for all published objects in the cluster. Objects that have been published to the orchestrator can be accessed by any transaction in the system, and are subject to migrations.

Workers interact with the global orchestrator in three cases: i) to publish a newly allocated object ii) to learn about the location of objects it has no knowledge of, passed as arguments to nanotransactions it was asked to execute, or iii) to request that the orchestrator pick a location to activate a nanotransaction. The last case would only occur if the worker on

which the nanotransaction was submitted or spawned does not know of the location of a subset of the target objects or if it knows that the dependencies are not already co-located. In this case, the orchestrator will have to engage in protocol to consolidate the set of data dependencies (§4.3).

3.3 Contention in Transaction Execution

Our local transaction scheduling policy allows us to omit local locks, while our distributed ownership model allows us to avoid distributed locks. Because conflicting nanotransactions are guaranteed to be processed serially by the same worker thread, there is no need for synchronization primitives in user code. Together, these decisions ensure that our system is deadlock free, by virtue of being lock-free.

We deliberately do not provide a guarantee of livelock freedom, as doing so would limit the expressivity of the programming model. In earlier designs we considered limiting the possible “conflict footprint” of nanotransactions (i.e. the number of objects they could receive as input) in order to make progress guarantees. Ultimately, because we intend Magpie to support arbitrary distributed applications and infrastructure, we abandoned these constraints. The additional power of epics comes with the sharp edge of the possibility of wasted work due to aborts: we choose to trade guaranteed progress for this expressivity. Programmers are permitted to implement any system they choose, including those (e.g., distributed cache coherence) that will not scale beyond a modest number of hosts.

Our nanotransaction scheduling policy converts what would be time spent waiting for a local lock into time that a task will have to spend in the queue of the local executor that owns the objects it needs to mutate. While wait times could be considerable for hot objects, it is no worse than the overheads associated with contested locks which exhibit unpredictable performance degradation [34]. An application that is fundamentally non-scalable because of its data access patterns will hit a performance wall no matter the approach taken to mutual exclusion.

As we demonstrated earlier (§2.2), our ownership model allows us to co-locate frequently accessed objects at the time they are first co-accessed, and amortize that cost by faster processing of subsequent accesses to overlapping sets of objects. While the ownership design affords us simplicity and performance in the construction of our transaction system, the requirement for co-accessed objects to be co-located means that programs are exposed to the potential of object thrashing under irregular access patterns. We consider this a policy problem, and something that our cost model (§4.3) must take into account.

3.4 Fault Tolerance

In addition to enforcing isolation of concurrent operations, ensuring top-down consistency requires that Magpie tolerate the partial failure of components. It must ensure that object

updates made by nanotransactions are *durable* across system restarts, that ownership changes are atomic even if interrupted by a node failure, and that no node represents a single point of failure that can compromise availability. We achieve these goals using standard mechanisms under realistic assumptions. In particular, we expect that system restarts are much more common than media failures or the permanent loss of hosts, and that failures in general are uncommon.

Standard redo logging (described in the next section) ensures durability of updates provided that hosts recover. Because objects fate-share with their owning worker, objects will be unavailable (even if copies exist elsewhere) while their owner is down. The flipside is that the system as a whole remains available for computation over live objects no matter how many hosts may be down. Magpie’s design is compatible with replication-based high availability techniques, though we do not explore them here.

Magpie’s guarantee of live object availability also requires that the ownership orchestrator not be a single point of failure. We achieve this by ensuring that in addition to objects, metadata regarding object ownership and placement state is also owned by the owning workers, and hence can be recovered by worker-to-worker communication if the orchestrator is unavailable. The orchestrator’s roles are thus reduced to that of a best-effort cache of this state to accelerate scheduling decisions, and of the coordinator of the ownership transfer protocol, which we describe in the next section.

4 Prototype Implementation

In this section we go over the details of our prototype implementation of Magpie in Rust, based on the system design we presented in the previous section. The full source code of Magpie can be found at <https://github.com/abenetopoulos/magpie-eurosys26>.

4.1 Implementing Magpie Programs

In our current prototype, users implement programs as Rust crates compiled to shared dynamic libraries, which are then loaded through `dlopen` by Magpie. We provide a set of macros with which programmers can annotate their data and function definitions to declare that they want them to be storable in objects and invocable through Magpie as nanotransactions, respectively. As an example, Fig. 4 shows the data model of a KVS implementation (§5.2) that consists of a static set of storage objects, along with the implementation of a nanotransaction for inserting or updating the value associated with a key.

Users annotate functions that should be invocable through Magpie using the `nandoize_lib` macro, which serves as a compiler plugin that generates the necessary code to resolve Magpie’s generic object references into concrete user data definitions as well as capturing state modifications as pre and post-image pairs to be used for transaction logging. A

nanotransaction is considered part of an epic if it includes a call to another nanotransaction in its body, using the macro `nando_spawn` and its variants. Fig. 5 shows how epics can be used to turn the previous static KVS into a resizable data store. Readers looking for more examples of nanotransaction invocation patterns are encouraged to refer to section §A.2 of the appendix.

We provide object-aware, persistent collection types to replace the standard library collections (PVec as the persistent variant of `Vec`, PHashMap as the persistent variant of `HashMap`, etc.), as mentioned in §3.1.1. Magpie does not rely on any specific feature of Rust to support execution of user programs, although Rust’s idiomatic distinction of mutable and immutable data maps well to Magpie’s ownership model.

4.2 Workers

On startup, a worker registers itself with the global ownership orchestrator and publishes its list of owned objects (if any) as well as their current versions. Once initialization is over, the worker can receive traffic.

Each worker exposes a web service that other workers or external clients can use to submit transaction execution requests. Workers communicate with each other and with the ownership orchestrator (§3.2.2) over gRPC (using Protobufs as the codec), while external clients can submit requests to workers through a JSON REST API. For the transaction execution subsystem (Fig. 3) we adopt a thread-per-core model, where all executor and scheduler threads, as well as the log manager thread, are pinned to their own cores.

To execute a request, a worker needs ownership of a nanotransaction’s object arguments. Each object is backed by a `mmap`-ed file that is updated in-place during transaction execution; read-only versions of objects are in-memory full copies of some of the most recent versions of a given object. Before calling a user function, an executor thread resolves any invariant references passed as arguments into concrete references to the target object. The executor thread is responsible for executing a nanotransaction to completion while tracking the changes it makes to existing persistent state. In case of a transaction abort, this information is used to clean up a nanotransaction’s partial effects, while in case of a commit it will be used to update a read-only version of the object to be made available to readers. In either case, the transaction’s execution status along with its effects are forwarded to the local log manager to ensure durability of updates (§3.4). Since our objects are multiversioned, transactions that only read a given object can be serviced by any executor thread.

A worker’s scheduler also consists of a set of one or more “completion” threads whose job is to process the results of executed transactions and take appropriate follow-up actions. For individual nanotransactions, this simply means notifying whoever submitted the nanotransaction of its outcome; for epics, these actions include handling the spawned tasks of

```

1 #[derive(Persistable)]
2 pub struct StorageBucket<V: Persistable> {
3     inner: PHashMap<PString, V>,
4 }

```

(a) Data Model

```

1 #[nandoize_lib]
2 pub fn put_internal<V: Persistable>(<
3     bucket: &mut StorageBucket<V>, key: String, value: V,
4 ) {
5     bucket.inner.insert(key, value);
6 }

```

(b) put nanotransactions

Fig. 4. Magpie program sample: The `Persistable` macro (Fig. 4a, line 1) is used to decorate user datatypes with convenience methods used by the runtime to extract images from the instantiated objects at runtime. A put operation is a nanotransaction, `put_internal`, that receives a bucket object reference as an argument, and performs a local insertion of the key/value pair on the given storage object.

```

1 #[derive(Persistable)]
2 pub struct RootBlock<V> {
3     bucket_object_ids: PVec<IPtr<StorageBucket<V>>>,
4 }
5
6 #[nandoize_lib]
7 pub fn put<V: Persistable>(<
8     root_block: &RootBlock<V>, key: String, value: V,
9 ) {
10     let bucket: IPtr = compute_bucket_object_id(root_block,
11         key);
12     nando_spawn_polymorphic!("kvs:put_internal::<V>", bucket,
13         key, value);
14 }

```

Fig. 5. Resizable KVS: We store a list of storage bucket pointers in a manifest object (`RootBlock`, lines 2-4), and call into `put_internal` with the pointer of the bucket that the target key hashes to (line 11), using a variant of rendezvous hashing as described in §5.2.

a given subtask, either submitting them locally or forwarding them to a remote worker, and propagating completion notifications across the spawned task graph.

Workers communicate with each other not only to schedule nanotransactions but also to coordinate ownership transfers and object migrations. To move objects between workers in the event of an ownership change, we leverage an `rsync` library to compute object signatures that are exchanged over RPC between the participating workers, and are subsequently compared and reconciled. The choice of `rsync` for this process was made to accelerate the prototyping phase, but we plan to explore alternative strategies for object data migration in the future.

4.3 Ownership Migration

Upon receiving a request to schedule an activation whose dependencies are not already co-located, the ownership orchestrator must compute an activation location to which all data dependencies should move by consulting its cost model. Our cost model is currently still under development. We chose to prioritize establishing mechanisms that enable fine-grained and low-overhead adaptivity, postponing the work of defining policy. The default policy favors minimizing the total number of object moves. Magpie can also be provided

with a *schedule*, which dictates a sequence of nanotransaction activations interleaved with data movement decisions made by the orchestrator. We use the schedule mechanism to explore different placement policies, and plan to incorporate what we learn in future enhancements of the orchestrator’s cost model.

For each object that is not already on the target host determined by the cost model, the ownership orchestrator will have to send a migration request to the given object’s current owner requesting that it be moved to the selected target host. Once the current owner acknowledges the start of the object migration, the ownership transfer proceeds in a peer-to-peer fashion between the workers involved, with the ownership orchestrator eventually being notified of the outcome of the ownership transfer. While ownership changes can be costly especially as the size and number of objects that need to be moved start to grow (§5.1.2), they always happen outside of the execution of user code, thereby avoiding the frequently surprising performance of DSM systems that perform implicit and on-demand data movement.

4.4 Fault Tolerance

Magpie ensures durability of changes via a logging mechanism: object mutations are made in place by the object’s owning worker, which also collects pre- and post-image pairs of all modified memory regions. These pairs, together with information about the transaction’s object read and write sets are packaged in a transaction record that is subsequently appended to the worker’s *local* log; the corresponding log entry is then flushed to a durable medium before being acknowledged to the caller. This ensures recoverability in the event of a worker restart, through a redo/undo mechanism [31]. Like objects, a worker’s log is unreplicated, which means that any modifications the worker made to its owned objects are lost in the event of a permanent node failure, and the objects owned by the failed node become unavailable.

The ownership orchestrator can fail at any time without impacting the correctness of applications. Because it represents soft-state, a failed orchestrator can always be replaced with a fresh one. Until a new orchestrator comes online, ownership state across the cluster is frozen, as workers cannot carry out ownership transfers entirely peer-to-peer.

The ownership transfer protocol involves a non-trivial state change on two workers, coordinated by the orchestrator. It is well-known that atomic commit protocols can wind up in an indeterminate state as a result of node failure [8] and our ownership transfer protocol is no exception. Using Alloy¹ [1], we established that even in the event of the concurrent failure of the orchestrator and a worker during an ownership change, only the objects owned by the failed worker become unavailable.

5 Experimental Evaluation

In evaluating Magpie, we aim to answer the following:

- **The Cost of Adaptivity:** What are the overheads of Magpie’s transaction execution subsystem (§5.1.1) and ownership transfer protocol (§5.1.2)?
- **The Benefits of Adaptivity:** Can Magpie’s programming abstractions be used to implement latency-sensitive distributed infrastructure that performs well under diverse workloads (§5.2)?
- **The Benefits of Being Unyielding:** How does Magpie perform for long-running tasks, as computation time starts to eclipse coordination overheads (§5.3)?

Our results show that Magpie can support both high-throughput, low-latency computations and longer batch processing jobs, performing competitively with the baselines.

Experimental Setup. We compared Magpie to Redis v7.4, Memcached v1.6.33, and Apache Spark 3.5.3. Unless otherwise specified, all systems are deployed on EC2 m7i.16 xlarge instances (32 cores / 64 threads, 256GB RAM) with SMT disabled, using EBS volumes for storage. We deployed Spark’s master on the same instance type; for single-host experiments, we co-located the Spark master and worker. All clients were running on the same instance types, but with SMT enabled.

Choice of Baselines. We think of Magpie as a system for building infrastructure *and* applications. For this reason, we chose to focus on two extremes in the design space of distributed systems in order to showcase Magpie’s generality.

On one extreme, we have low-latency, high-throughput infrastructure which will exercise our overheads. We decided to compare against two fast in-memory KVSs: Redis, which exhibits good single-host performance but is not designed to scale, and Memcached which can scale out but has a radically different architecture. By showing how a single KVS implementation in Magpie is competitive with both, we illustrate the performance and flexibility of our adaptive runtime.

On the other extreme, we have data processing systems that accelerate computations by scaling up. Spark is a ubiquitous, state-of-the-art large-scale data processing system, but it also exemplifies the ethos of “doing more with more”,

exhibiting parallelizable overheads. By showing a Magpie implementation perform better on modest hardware configurations, we showcase the success of our design in letting us do “more with less”.

5.1 Microbenchmarks

In this section we quantify the cost of adaptivity by measuring the overheads of Magpie’s basic mechanisms: recoverable transaction execution and ownership transfer.

5.1.1 Transaction Execution. We first measure the overheads of Magpie’s transaction execution subsystem as described in §4.2, by submitting 1000 noop transactions to a Magpie worker running with a single executor; table 1 shows a breakdown of mean latency by component. The mean service latency for these transactions is $12.25\mu\text{s}$ ($\pm 0.33\mu\text{s}$), with the unaccounted for time being spent off-CPU with tasks in queues between components or propagating completion notifications to the transaction manager. We deem this performance acceptable, especially given our ability to increase throughput by increasing the number of worker threads.

Component	Latency (μs)	Object Size	Latency (ms)
Task Submission	0.83 (± 0.30)	100kB	5.41 (± 0.04)
Task Completion	1.67 (± 0.02)	1MB	13.46 (± 0.05)
Execution	0.55 (± 0.006)	10MB	147.1 (± 0.59)
Logging	0.23 (± 0.002)	100MB	1503.52 (± 2.23)

Table 1. Base Overheads

Table 2. Transfer Latency

5.1.2 Ownership Protocol. We proceed to evaluate the impact of different object sizes to the latency of our object ownership change protocol, along with the overheads of object migration, and present the results in table 2.

We observe an almost linear correlation between object transfer latency and object size, with latencies around 150ms for modestly sized objects. For reference, the KVS objects resulting from the benchmarks in §5.2 rarely exceeded 10MB in size; the only objects that have exceeded that size while we have been developing Magpie have been graph-related objects, which tend to be around 200-250MB in size.

The latencies presented here represent the worst case scenario – moving an object to a host for the first time. If the destination host previously owned an object, then we benefit from rsync’s delta encoding. Additionally, the impact of object migration is reduced by preferring to ship nanotransactions to their data dependencies, relying on object migrations only when an operation’s data dependencies are not already co-located, as described in §3.2.2.

¹<https://github.com/abenetopoulos/ownership-model-eurosys26>

5.2 Use Case: Distributed Key/Value Store

Given Magpie's performance characteristics as shown in the previous section, in this experiment we tried to answer the following question: would Magpie be a good platform on top of which to implement a low-latency, high-throughput piece of distributed infrastructure like a key/value store? The experimental results in this section answer this question affirmatively. Additionally, they show that Magpie's ability to dynamically adapt the placement of compute and data leads to imperceptible performance regressions compared to our baselines (between 0.01-0.1% for Magpie compared to a 29% increase in latency for Redis and 46% for Memcached) as the application workload shifts from blind writes to a more realistic read/modify/write (RMW) workload, highlighting the issues posed by traditionally rigid system designs (§2). For this evaluation, we consider the static version of the KVS described in §4.

Clients issuing commands against the KVS run a variant of rendezvous hashing [39] (using storage bucket object IDs in place of server addresses) to determine the invariant pointer to the storage bucket for a key. This pointer is passed as an argument to the nanotransaction that performs the concrete requested operation (e.g. `put_internal` in Fig. 4b).

Workloads consist of single-key operations in a mix of reads and either blind updates or a more realistic RMW workload, performing a predicate-based update of existing values. The keyspace for all experiments consists of 10M distinct keys, sampled uniformly. We issue a read-mostly workload of 90/10% reads to updates, and a balanced workload of 50/50% reads and writes for 30s without pre-loading. We use variable-length keys up to 12 bytes long, and 4-byte integer values.

We find that the overheads of Magpie are low enough that it can be used for latency-critical infrastructure, achieving between 1.25-1.6x higher peak throughput than Redis and 1.02-1.48x higher peak throughput than Memcached.

5.2.1 Single-node Performance. Fig. 6 shows a comparison of the mean throughput and latency of both Magpie and Redis running on a single host, as we scale concurrent clients from 1 to 256. We run Magpie both with a single worker thread (`magpie-kv(single-threaded)`), to emulate Redis, and also with 16 worker threads (`magpie-kv(multi-threaded)`), without making any modifications to the KVS source code.

When both systems are unsaturated, Magpie's increased consistency and isolation guarantees lead to 35-40% higher mean latency than that of Redis. However, the single-threaded variant's peak throughput is 50-60% higher than Redis', owing to Magpie's fast read path. Magpie benefits from being able to execute KVS commands using multiple worker threads and gets saturated much later than the baseline, achieving almost 3x the peak throughput of Redis and 2-3x the throughput of the single-threaded variant. This ability to transparently leverage the available parallelism of a single

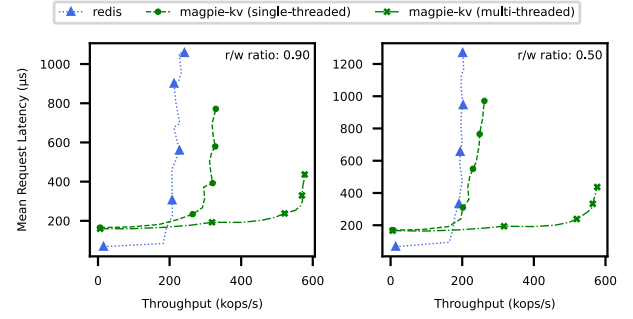


Fig. 6. magpie-kv vs Redis: Performance for 1-256 closed-loop clients for two different get/put workload mixes, 90/10% (left) and 50/50% (right).

host illustrates one aspect of Magpie's adaptivity, namely its ability to scale up user programs without any rewrites.

Fig. 7 shows the results of running a RMW workload using the same parameters as before. Using simple Redis commands for this workload exposes clients to races in the case of concurrent updates to the same key; for this reason, we also evaluated the performance of Redis transactions, which are used to execute multiple operations in a single step and provide mutual exclusion for the accessed keys, at a heavy performance cost.

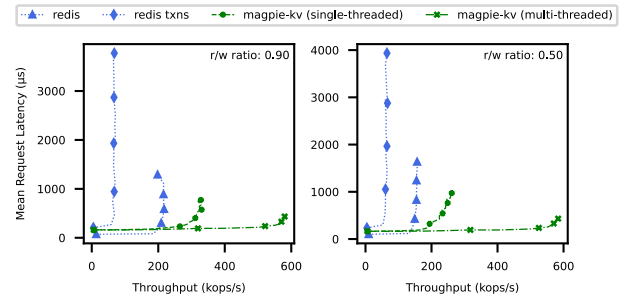


Fig. 7. magpie-kv vs Redis: Performance for 1-256 closed-loop clients for a RMW workload with 90/10% (left) and 50/50% (right) reads vs updates.

This experiment shows the benefit of Magpie's data-driven placement mechanisms, allowing it to execute the entire RMW sequence entirely on the server that owns the target bucket object with a single round-trip to the client. In contrast, the Redis-based variant incurs additional round-trips, since the values associated with the target keys need to be shipped back to the clients for the conditional update to take place §2. As a result, Redis' peak throughput drops by 25% as the ratio of writes to reads increases even when payload sizes are extremely small – as payloads increase in size, the effect will be more pronounced.

5.2.2 Multi-node Performance. For this experiment, we compare Magpie’s performance to that of **Memcached** when each is running on four hosts. To more closely emulate Memcached and provide a more direct performance comparison, we distribute Magpie’s bucket objects between the worker hosts ahead of time and use “fat” clients that have knowledge of the placement of available bucket objects, directly submitting commands to their target bucket’s owner.

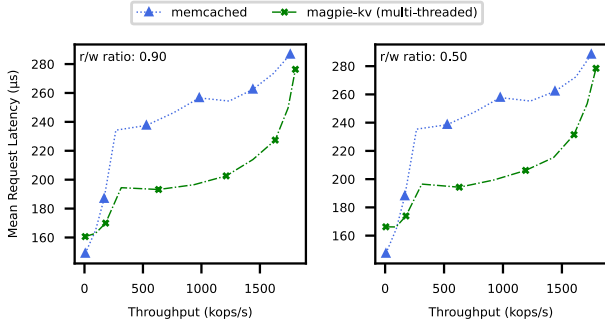


Fig. 8. magpie-kv vs Memcached (4 hosts) Performance for 1-256 closed-loop clients for two different get/put workload mixes, 90/10% (left) and 50/50% (right).

Fig. 8 shows a comparison of the mean throughput and latency exhibited by Magpie compared to Memcached, each running with 16 worker threads per host, for 1 to 512 concurrent clients issuing simple get/put commands. Even though Memcached exhibits much lower latencies for low numbers of clients, Magpie handles the increase in offered load much more gracefully, achieving 2.5% higher peak throughput for the simple get/put workload. We observe similar results for the RMW workload (shown in Fig. 9), with Magpie achieving almost 90% higher peak throughput thanks to a reduced number of client-server round-trips owing to its data-oriented placement decisions.

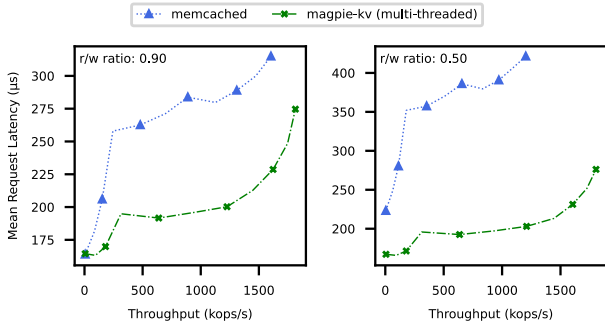


Fig. 9. magpie-kv vs Memcached (4 hosts): Performance for 1-256 closed-loop clients for a RMW workload with 90/10% (left) and 50/50% (right) reads vs updates.

This experiment illustrates the benefits of Magpie’s late-binding approach to placement, enabling best-effort scheduling decisions. Clients need only have knowledge of the set of bucket objects available to them, but do not necessarily need to be aware of how their placement changes in the cluster or even of the entire set of workers. If they send their request to the wrong worker, it will get forwarded to the bucket’s owning host whose identity will be contained in the reply they receive so that they can cache the mapping if they choose to. Similarly, the placement and layout of KVS objects in the cluster can change without requiring any modifications to the implementation of the KVS access methods, as appropriate routing is facilitated “outside” of user programs by Magpie’s ownership mechanism.

5.3 Use Case: Parallel Data Processing

We now switch our attention to longer-running workflows that digest large amounts of data. We demonstrate that workflows that exhibit articulations at the points in a program where there is potential data parallelism to be “discovered” can perform well on a single host, and scale up to larger numbers of hosts for larger problems.

5.3.1 Iterative Parallel Computations. We implemented a library of parallelizable graph algorithms (called nano4r) from the literature, and here we discuss the performance of an adaptive implementation of PageRank [11, 28] (§5.3.1), evaluated over real-world graphs of varying sizes. All graph datasets were sourced from the SNAP library [3]. We compared our implementations against GraphX [42], another library that provides graph-specific abstractions over a general-purpose distributed runtime (in this case Apache Spark).

We find that Magpie running on a single host always outperforms Spark running on the same setup even on modestly sized graphs, with Magpie’s single-host performance frequently remaining competitive even when Spark is given more resources. We also find that Magpie can scale competitively with Spark.

For both systems, we partition the target graph into n partitions (specified by the user during parsing) following a random partitioning strategy — in the case of Magpie, the goal of the strategy is to split the vertices of the underlying graph in a roughly uniform way across the n partitions, modeled as adjacency lists. For the multihost experiments for Magpie, we move the partition objects after parsing in such a way so that each host owns the same number of partitions before we execute the algorithm.

Our implementation of PageRank follows the bulk synchronous parallel model [40], performing a set of “supersteps” with explicit barriers between them, until a convergence condition is met. A more detailed explanation of the structure of the PageRank epic is presented in section §A.1 of the appendix.

Workload	nano4r (Magpie)				GraphX (Spark)			
	1 Host	2 Hosts	4 Hosts	8 Hosts	1 Host	2 Hosts	4 Hosts	8 Hosts
eu-email (265k Vertices, 420k Edges)	2.23	11.48	13.17	18.03	12.87	25.54	27.67	23.10
soc-liveJournal (4.8M Vertices, 68.9M Edges)	215.75	132.54	123.51	119.06	307.37	179.10	120.79	106.80

Table 3. End-to-end execution latency (sec): For each system and workload, we have marked their best performance in bold.

We compared our PageRank implementation with GraphX’s Pregel-based [27] PageRank implementation, and ran both to convergence with a threshold of 10^{-7} over two graphs from the SNAP library, with results in table 3. During each superstep, graph partitions are processed in parallel. In the beginning there are as many parallel nanotransactions as there are partitions in the graph; as partitions reach convergence, the required parallelism diminishes.

For the smaller graph, eu-email, both systems achieve their best performance on a single host, with Magpie being 5.7x faster than Spark. Increasing the number of hosts makes Magpie an order of magnitude slower, but still competitive with Spark’s best performance. In Magpie’s case, this performance degradation stems from the fact that the graph is small enough that most nanotransactions execute fast enough that the network communication in each superstep introduces frequent stalls that dominate the computation’s latency.

For the larger graph, soc-liveJournal, Magpie’s single-host performance once again exceeds Spark’s, completing the computation 1.4x faster. Unlike with the smaller graph, task durations are meaningfully high that they offset the network latency of exchanging intermediate results between the intra and inter-partition tasks of each superstep, leading to a speedup of 1.81x as we increase the number of Magpie workers, compared to Spark’s 2.88x speedup.

The performance difference between Spark and Magpie can be attributed in part to the different performance characteristics of the implementation languages of the two systems (Java versus Rust, respectively). For small clusters where the total available parallelism is relatively low, each Spark executor needs to process tasks with relatively large inputs, slowing down execution. However, another major factor contributing to Magpie’s improved performance is its judicious use of the network. Fig. 10 shows the cumulative traffic between workers of each system for the duration of a PageRank computation on two hosts. Spark/GraphX workers perform a series of label propagation stages, transferring large amounts of data between supersteps as messages along the graph’s edges [27]. GraphX’s built-in vertex cut partitioning strategy [2] is disadvantageous here, since messages need to be aggregated based on the corresponding vertex whose edges might be spread across workers. In contrast, the adjacency list model we adopted in nano4r leads to exchanges of small

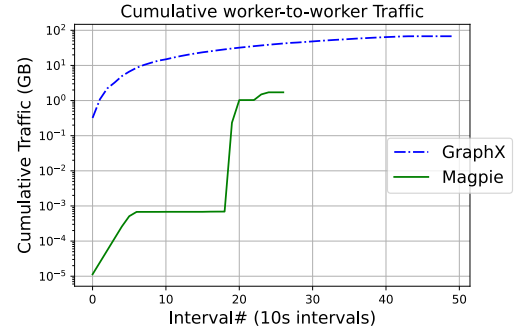


Fig. 10. Cumulative network bandwidth (log scale): The large spike in traffic around the 180-200s mark is attributable to the initial exchange of per-partition state, the worst-case scenario for data transfer (§5.1.2).

amounts of data in the middle of each superstep to propagate information along cross-partition edges.

5.3.2 Data-Parallel Computations. In this experiment, we evaluated the performance of a parallel implementation of the Smith-Waterman sequence alignment algorithm [37]. We consider a block-based approach where the original $n \times n$ matrix is divided into a square matrix of $m \times m$ blocks. The blocks that belong to the same antidiagonal of the matrix can be computed in parallel to each other, since the only data dependencies of a given block reside in the two previous antidiagonals. The number of blocks directly affects the available parallelism to be exploited, but unlike PageRank the potential parallelism fluctuates during execution.

We implemented Smith-Waterman as an epic that receives two input strings and a number of blocks to split each string into. The epic spawns a nanotransaction for each of the blocks in the matrix; as tasks’ data dependencies are met, they get scheduled. Magpie takes an incremental approach to scheduling – if the number of available tasks all fit on a single worker, Magpie performs all work locally. As the number of available tasks starts exceeding the capacity of a worker, Magpie incrementally sheds load to the next worker. When scheduling tasks on multiple hosts, Magpie also prefers to co-locate dependent tasks, to minimize data movement. Fig. 11 presents the parallel speedup achieved by Magpie as we vary both the size of the input strings from 128kB to 1MB and the number of available hosts from 1 to 12. For this experiment,

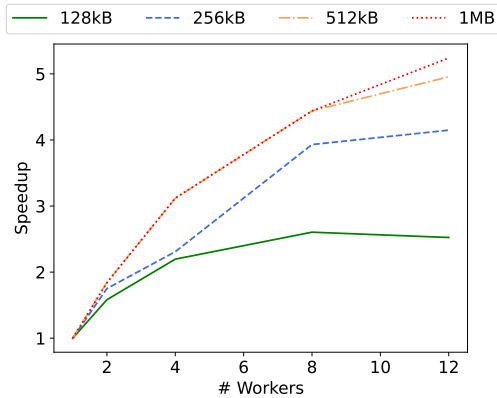


Fig. 11. Speedup of Smith-Waterman: This figure shows the speedup achieved by Magpie as the number of available workers is increased and the problem size is varied. For each point, we show the highest speedup achieved across different matrix block sizes.

Magpie workers were executing on Cloudlab [15] c6525-25g hosts (32 threads, 128GB RAM).

The choice of m when invoking the epic is key to achieving good performance. For smaller problem sizes, we can see that our system quickly reaches a point of diminishing returns (similar to the PageRank case), as increasing the block count makes individual tasks execute very fast, turning communication and data transfer overheads between tasks the dominant latency component. For larger problems which we decompose into tasks of more substantial execution time (therefore amortizing protocol overheads), the knee of the speedup curves moves further to the right, and Magpie makes better use of available resources.

6 Related Work

Our ownership model and scheduling approach resemble that of Ray [32], from which Magpie draws inspiration. Unlike Ray, our programming model does not rely on immutable data, and presents a simpler model of data and computation. In Magpie there is no need to distinguish between lightweight tasks and stateful actors; the actor pattern is just a scheduling strategy in which a set of objects remains owned by one host for substantially long, where all activations are shipped and processed sequentially.

Nanotransactions superficially resemble Sinfonia’s minitransactions [5]. Like a minitransaction, a nanotransaction makes it possible to execute an atomic compound operation across multiple data structures that may be located on multiple physical hosts. Unlike a minitransaction, a nanotransaction never requires a costly two-phase commit protocol to ensure atomic visibility of its changes; instead, it relies on transparent data migration to ensure that updated objects are first co-located on a single host.

Magpie’s concept of data mobility is inspired by Emerald [21]. Unlike Emerald, we eschew the concept of “locatics” as we believe the runtime is in a better position to dictate placement than programmers.

Magpie’s epic task graphs take inspiration from CIEL’s dataflow abstractions [33], but unlike CIEL Magpie supports safe in-place data mutation thanks to its top-down consistency guarantees.

Far memory systems [14, 23, 35] share a lot of the same goals as Magpie, allowing programs to use remote memory as another level of the memory hierarchy, but fail to provide the level of adaptivity that is key to Magpie’s performance — FaRM’s consistent hashing prevents it from adapting data placement to minimize coordination; in Zeus, an ownership system facilitates repartitioning, but programs are monolithic processes that submit transaction requests to Zeus’ load balancers, failing to take advantage of fluid and late-bound computation.

7 Conclusion

Through Magpie’s programming model, programmers can exploit cluster resources while remaining agnostic to placement concerns, by expressing the *logical* boundaries of local compute using nanotransactions and objects. The disparate applications we used to validate Magpie highlight its generality, being equally well-suited to programming systems that must perform small data modifications quickly as to applications that process massive amounts of data, with its adaptivity and focus on uninterrupted compute providing improved performance and high utilization of resources. Meanwhile, it was able to provide a serializability guarantee — the gold standard of correctness in databases but typically avoided in general purpose systems — as a default with modest costs, freeing programmers from reasoning about concurrency.

Acknowledgments

We would like to thank Devashish Purandare, Konstantinos Kallas, Daniel Bittman, Shel Finkelstein, and Ike Nassi for their feedback on early drafts of this work. We would also like to thank our shepherd, Gerd Zellweger, and the anonymous reviewers for their comments and suggestions that helped improve this paper. This work is supported by the National Science Foundation under Grant No. CNS-2212235.

References

- [1] Alloy Analyzer. <https://alloytools.org/>.
- [2] GraphX - Optimized Representation. <https://spark.apache.org/docs/latest/graphx-programming-guide.html#optimized-representation>.
- [3] Stanford Network Analysis Project. <https://snap.stanford.edu/>.
- [4] A. Adya, D. Myers, J. Howell, J. Elson, C. Meek, V. Khemani, S. Fulger, P. Gu, L. Bhuvanagiri, J. Hunter, R. Peon, L. Kai, A. Shraer, A. Merchant, and K. Lev-Ari. Slicer: Auto-Sharding for Datacenter Applications. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pages 739–753, Nov. 2016.

- [5] M. K. Aguilera, A. Merchant, M. Shah, A. Veitch, and C. Karamanolis. Sinfonia: A New Paradigm for Building Scalable Distributed Systems. *ACM Trans. Comput. Syst.*, 27(3), Nov 2009.
- [6] J. K. Bennett, J. B. Carter, and W. Zwaenepoel. Munin: Distributed Shared Memory Based on Type-Specific Memory Coherence. In *Proceedings of the Second ACM SIGPLAN Symposium on Principles & Practice of Parallel Programming*, PPOPP '90, page 168–176, 1990.
- [7] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, and P. O'Neil. A Critique of ANSI SQL isolation levels. *SIGMOD Rec.*, 24(2):1–10, May 1995.
- [8] P. A. Bernstein, V. Hadzilacos, and N. Goodman. *Concurrency control and recovery in database systems*. Addison-Wesley Longman Publishing Co., Inc., USA, 1987.
- [9] D. Bittman, P. Alvaro, P. Mehra, D. D. E. Long, and E. L. Miller. Twizzler: A Data-Centric OS for Non-Volatile Memory. *ACM Trans. Storage*, 17(2), jun 2021.
- [10] D. Bittman, R. Soulé, E. L. Miller, V. Shrivastav, P. Mehra, M. Boisvert, A. Silberschatz, and P. Alvaro. Don't Let RPCs Constrain Your API. In *Proceedings of the Twentieth ACM Workshop on Hot Topics in Networks*, HotNets '21, page 192–198, 2021.
- [11] S. Brin and L. Page. The anatomy of a large-scale hypertextual Web search engine. *Computer Networks and ISDN Systems*, 30(1):107–117, 1998. Proceedings of the Seventh International World Wide Web Conference.
- [12] S. Bykov, A. Geller, G. Kliot, J. Larus, R. Pandya, and J. Thelin. Orleans: Cloud Computing for Everyone. In *ACM Symposium on Cloud Computing (SOCC 2011)*, Oct. 2011.
- [13] J. Dean and S. Ghemawat. MapReduce: Simplified Data Processing on Large Clusters. In *OSDI'04: Sixth Symposium on Operating System Design and Implementation*, pages 137–150, 2004.
- [14] A. Dragojević, D. Narayanan, O. Hodson, and M. Castro. Farm: fast remote memory. In *Proceedings of the 11th USENIX Conference on Networked Systems Design and Implementation*, NSDI'14, page 401–414, 2014.
- [15] D. Duplyakin, R. Ricci, A. Maricq, G. Wong, J. Duerig, E. Eide, L. Stoller, M. Hibler, D. Johnson, K. Webb, A. Akella, K. Wang, G. Ricart, L. Landweber, C. Elliott, M. Zink, E. Cecchet, S. Kar, and P. Mishra. The Design and Operation of Cloudlab. In *Proceedings of the 2019 USENIX Conference on Usenix Annual Technical Conference*, USENIX ATC '19, page 1–14, USA, 2019. USENIX Association.
- [16] B. Fan, D. G. Andersen, M. Kaminsky, and M. D. Mitzenmacher. Cuckoo Filter: Practically Better Than Bloom. In *Proceedings of the 10th ACM International Conference on emerging Networking Experiments and Technologies*, page 75–88, Sydney Australia, Dec. 2014. ACM.
- [17] H. Garcia-Molina, D. Gawlick, J. Klein, K. Kleissner, and K. Salem. Coordinating multi-transaction activities. Technical report, Stanford University, 1990.
- [18] D. R. Hanson. Fast allocation and deallocation of memory based on object lifetimes. *Software: Practice and Experience*, 20(1):5–12, 1990.
- [19] C. Hewitt, P. Bishop, and R. Steiger. A universal modular ACTOR formalism for artificial intelligence. In *Proceedings of the 3rd International Joint Conference on Artificial Intelligence*, IJCAI'73, 1973.
- [20] L. Hoang Le, E. Fynn, M. Eslahi-Kelozazi, R. Soulé, and F. Pedone. DynaStar: Optimized Dynamic Partitioning for Scalable State Machine Replication. In *2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*, pages 1453–1465, 2019.
- [21] E. Jul, H. Levy, N. Hutchinson, and A. Black. Fine-grained mobility in the Emerald system. *ACM Trans. Comput. Syst.*, 6(1):109–133, feb 1988.
- [22] R. Kallman, H. Kimura, J. Natkins, A. Pavlo, A. Rasin, S. Zdonik, E. P. C. Jones, S. Madden, M. Stonebraker, Y. Zhang, J. Hugg, and D. J. Abadi. H-Store: A High-Performance, Distributed Main Memory Transaction Processing System. *Proc. VLDB Endow.*, 1(2):1496–1499, aug 2008.
- [23] A. Katsarakis, Y. Ma, Z. Tan, A. Bainbridge, M. Balkwill, A. Dragojevic, B. Grot, B. Radunovic, and Y. Zhang. Zeus: locality-aware distributed transactions. In *Proceedings of the Sixteenth European Conference on Computer Systems*, EuroSys '21, page 145–161, 2021.
- [24] P. Keleher, A. L. Cox, S. Dwarkadas, and W. Zwaenepoel. Tread marks: Distributed shared memory on standard workstations and operating systems. In *USENIX Winter 1994 Technical Conference (USENIX Winter 1994 Technical Conference)*, Jan. 1994.
- [25] H. Lim, D. Han, D. G. Andersen, and M. Kaminsky. MICA: A holistic approach to fast in-memory key-value storage. In *Proc. 11th USENIX NSDI*, Apr. 2014.
- [26] S. Luo, H. Xu, C. Lu, K. Ye, G. Xu, L. Zhang, Y. Ding, J. He, and C. Xu. Characterizing microservice dependency and performance: Alibaba trace analysis. In *Proceedings of the ACM Symposium on Cloud Computing*, SoCC '21, page 412–426, 2021.
- [27] G. Malewicz, M. H. Austern, A. J. Bik, J. C. Dehnert, I. Horn, N. Leiser, and G. Czajkowski. Pregel: a system for large-scale graph processing. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data*, SIGMOD '10, page 135–146, 2010.
- [28] B. Manaskasemsak and A. Rungsawang. An Efficient Partition-Based Parallel PageRank Algorithm. In *11th International Conference on Parallel and Distributed Systems (ICPADS'05)*, volume 1, pages 257–263 Vol. 1, 2005.
- [29] F. McSherry, M. Isard, and D. G. Murray. Scalability! But at what COST? In *15th Workshop on Hot Topics in Operating Systems (HotOS XV)*, May 2015.
- [30] E. L. Miller, G. Neville-Neil, A. Benetopoulos, P. Mehra, and D. Bittman. Pointers in Far Memory. *Commun. ACM*, 66(12):40–45, Nov. 2023.
- [31] C. Mohan, D. Haderle, B. Lindsay, H. Pirahesh, and P. Schwarz. Aries: a transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging. *ACM Trans. Database Syst.*, 17(1):94–162, Mar. 1992.
- [32] P. Moritz, R. Nishihara, S. Wang, A. Tumanov, R. Liaw, E. Liang, M. Elibol, Z. Yang, W. Paul, M. I. Jordan, and I. Stoica. Ray: A Distributed Framework for Emerging AI Applications. In *Proceedings of the 13th USENIX Conference on Operating Systems Design and Implementation*, OSDI'18, page 561–577, 2018.
- [33] D. G. Murray, M. Schwarzkopf, C. Smowton, S. Smith, A. Madhavapeddy, and S. Hand. CIEL: a universal execution engine for distributed data-flow computing. In *Proceedings of the 8th USENIX Conference on Networked Systems Design and Implementation*, NSDI'11, page 113–126, USA, 2011. USENIX Association.
- [34] K. K. Pusukuri, R. Gupta, and L. N. Bhuyan. Shuffling: a framework for lock contention aware thread scheduling for multicore multiprocessor systems. In *Proceedings of the 23rd International Conference on Parallel Architectures and Compilation*, PACT '14, page 289–300, New York, NY, USA, 2014. Association for Computing Machinery.
- [35] Z. Ruan, M. Schwarzkopf, M. K. Aguilera, and A. Belay. AIFM: High-Performance, Application-Integrated far memory. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 315–332, Nov. 2020.
- [36] K. Seemakhupt, B. E. Stephens, S. Khan, S. Liu, H. Wassel, S. H. Yeganeh, A. C. Snoeren, A. Krishnamurthy, D. E. Culler, and H. M. Levy. A cloud-scale characterization of remote procedure calls. In *Proceedings of the 29th Symposium on Operating Systems Principles*, SOSP '23, page 498–514, 2023.
- [37] T. Smith and M. Waterman. Identification of common molecular subsequences. *Journal of Molecular Biology*, 147(1):195–197, 1981.
- [38] A. Sriraman and A. Dhanotia. Accelerometer: Understanding acceleration opportunities for data center overheads at hyperscale. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '20, page 733–750, 2020.
- [39] D. Thaler and C. V. Ravishankar. A Name-Based Mapping Scheme for Rendezvous. *Electrical Engineering and Computer Science Department, The University of Michigan, Ann Arbor, Michigan*, pages 1–31, 1996.

- [40] L. G. Valiant. A Bridging Model for Parallel Computation. *Commun. ACM*, 33(8):103–111, Aug. 1990.
- [41] S. Wang, E. Liang, E. Oakes, B. Hindman, F. S. Luan, A. Cheng, and I. Stoica. Ownership: A Distributed Futures System for Fine-Grained Tasks. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, pages 671–686, Apr. 2021.
- [42] R. S. Xin, J. E. Gonzalez, M. J. Franklin, and I. Stoica. GraphX: a resilient distributed graph system on Spark. In *First International Workshop on Graph Data Management Experiences and Systems, GRADES '13*, 2013.
- [43] Y. Yang, M. Youill, M. Woicik, Y. Liu, X. Yu, M. Serafini, A. Aboulnaga, and M. Stonebraker. FlexPushdownDB: hybrid pushdown and caching in a cloud DBMS. *Proc. VLDB Endow.*, 14(11):2101–2113, July 2021.
- [44] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica. Spark: Cluster Computing with Working Sets. In *2nd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 10)*, June 2010.
- [45] Q. Zhang, X. Chen, S. Sankhe, Z. Zheng, K. Zhong, S. Angel, A. Chen, V. Liu, and B. T. Loo. Optimizing Data-intensive Systems in Disaggregated Data Centers with TELEPORT. In *Proceedings of the 2022 International Conference on Management of Data*, SIGMOD '22, page 1345–1359, New York, NY, USA, 2022. Association for Computing Machinery.

APPENDIX

A.1 Epic Structure Example: PageRank

As discussed in §3.1.4, epics are a dynamic dataflow abstraction. At compilation time, we can derive the call graph of any epic; we refer to this call graph as the epic’s *logical plan*. This plan encodes all spawner/spawnee relations, as well as the potential data and control dependencies between tasks; Fig. 12 shows the logical plan of the PageRank epic. The dataflow graph that results from the execution of a given epic over concrete data is referred to as the epic’s *physical plan*; Fig. 18 shows an example of the physical plan for a graph with two partitions, for a single superstep of PageRank.

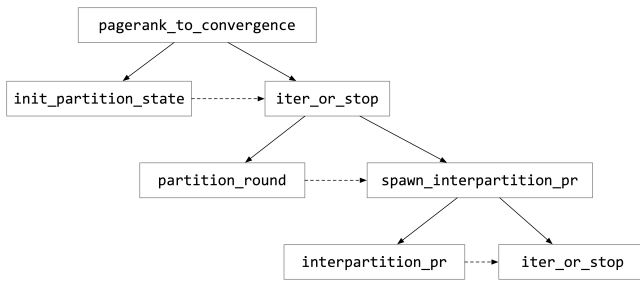


Fig. 12. PageRank Logical Plan: Horizontal dashed arrows represent intertask dependencies (data or control), while solid arrows represent a “spawns” relationship between tasks.

The epic starts by spawning rank state initialization tasks (`init_partition_state`) for each vertex in a given partition. Subsequently, every round of the algorithm proceeds in two steps: first, the task `partition_round` checks for convergence of individual vertices in a partition. If a vertex is deemed converged, it is ignored; otherwise, its rank at the end of the previous iteration is stored as its previous rank, and its new rank is computed. This computation might yield partial results for vertices with incoming edges from remote partitions; these partial results will subsequently be updated to their final value through a series of interpartition tasks. Between these intra and interpartition tasks we introduce a synchronization step, `spawn_interpartition_pr`, which is scheduled once all intra partition tasks for the current superstep have finished executing. This step is essential to the performance of our implementation, as it enables partition-wise parallelism of interpartition tasks, since the information needed to update vertex ranks (i.e. the rank of all foreign vertices at the end of the previous round) is guaranteed to be up-to-date and not change until the beginning of the next iteration, allowing all interpartition tasks to use read-only versions of the per-partition rank states.

Once all interpartition tasks have finished executing, we check if we’ve reached convergence for all vertices in the

graph by executing `iter_or_stop`; if so, the algorithm terminates, otherwise a new round of intra and interpartition tasks is spawned.

A.2 Graph Code Sample

In Fig. 13 we have listed a subset of the data definitions of our graph library `nano4r`. We model graphs as a manifest object that contains invariant pointers to a set of partition objects (lines 2-5), which are adjacency lists for the partition’s vertices (lines 9-11). In addition to the partition pointers, the manifest object also maintains a cuckoo filter [16] per partition (line 3), allowing any nanotransaction that has access to the graph manifest to conditionally spawn tasks predicated on the presence of a vertex in a partition without accessing the target partition object. The overhead of the tasks being spawned due to false-positive results are negligible, as they return immediately.

```

1  #[derive(Persistable)]
2  struct Graph {
3      filters: PVec<PCuckooFilter>,
4      partitions: PVec<TypedIPtr<GraphPartition>>,
5      /* ... */
6  }
7
8  #[derive(Persistable)]
9  struct GraphPartition {
10     adjacencies: PHashMap<VertexId, PVec<VertexId>>,
11 }
  
```

Fig. 13. Graph Data Model

We also present the code for a naive DFS algorithm implemented using epics, with the corresponding logical plan shown in Fig. 14.

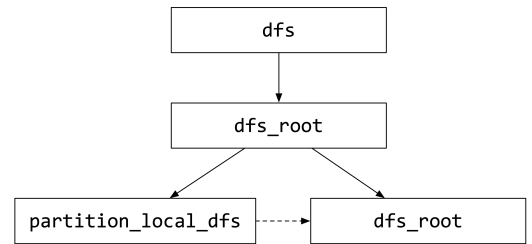


Fig. 14. DFS Logical Plan

The entry point for the DFS algorithm is the function shown in Fig. 15, which initializes a search state object (lines 3-6) before using the `nano_spawn!` macro to call `dfs_root` and kick off the search procedure.

As seen in Fig. 16, the job of this function is to pick an unvisited vertex (if one exists) as a search root (lines 5-7) and then invoke the search procedure starting from that root by calling `partition_local_dfs`. Sometimes we want

```

1 #[nandoize_lib]
2 fn dfs(graph: &Graph) -> IPtr {
3     let search_state = allocate_and_init!(
4         object_tracker, DFSSearchState,
5     );
6     search_state.initialize_search_state(graph);
7
8     nando_spawn!(
9         "nano4r::dfs_root",
10        iptr_of!(graph),
11        iptr_of!(search_state),
12    );
13
14    search_state_iptr
15 }

```

Fig. 15. Algorithm entry point

to launch a task only after another task (or collection of tasks) has finished executing — in those cases we use the `nando_yield_sink!` macro as in lines 25-29 of Fig. 16. This macro lets users specify a set of upstream control dependencies that need to complete before another task may begin executing. This is the mechanism we use to set up barriers for `spawn_interpartition_pr` and `iter_or_stop` in the PageRank computation (Fig. 12).

Each invocation of `partition_local_dfs` (code for which is shown in Fig. 17) only proceeds as long as the vertex v it is considering is stored in the graph partition passed to it as an argument; if that is not the case, it invokes itself with the target partition as an argument (lines 19-34). Once the search stack is empty, the task subgraph that was rooted in the invocation of `partition_local_dfs` from `dfs_root` is considered complete, and a notification is sent to the pending sink task of `dfs_root` which will cause it to be scheduled for execution.

```

1 #[nandoize_lib]
2 fn dfs_root(
3     graph: &Graph, search_state: &DFSSearchState,
4 ) {
5     let root_vertex = search_state.get_unvisited_vertex();
6     let Some(root_vertex) = root_vertex else {
7         return;
8     };
9     for idx in 0..graph.partitions.len() {
10        if !graph.filters[idx].contains(root_vertex) {
11            continue;
12        }
13        // Spawn partition-local search.
14        let partition_ptr = graph.partitions[idx];
15        let graph_iptr = iptr_of!(graph);
16        let search_state_iptr = iptr_of!(search_state);
17        let partition_task = nando_spawn!(
18            "nano4r::partition_local_dfs",
19            graph_iptr,
20            partition_ptr,
21            root_vertex,
22            search_state_iptr
23        );
24        // After search from the current root is done,
25        // call this function to pick a new root.
26        nando_yield_sink!(
27            "nano4r::dfs_root",
28            &[partition_task],
29            graph_iptr,
30            search_state_iptr,
31        );
32    }
33 }

```

Fig. 16. Search Root Selection

```

1  #[nandoize_lib]
2  fn partition_local_dfs(
3      graph: &Graph,
4      partition: &GraphPartition,
5      proot_vertex: VertexId,
6      state: &mut DFSSearchState,
7  ) {
8      if !partition.adjacencies.contains(&proot_vertex) {
9          return;
10     }
11     state.stack.push(proot_vertex);
12     while !state.stack.is_empty() {
13         let v = state.stack.pop().unwrap();
14         if state.is_discovered(v) {
15             continue;
16         }
17         // Cannot continue exploring further, need to spawn
18         // on another object
19         if !partition.adjacencies.contains(&v) {
20             for idx in 0..graph.partitions.len() {
21                 if !graph.filters[idx].contains(v) {
22                     continue;
23                 }
24                 let partition_ptr = graph.partitions[idx];
25                 nando_spawn!(
26                     "nano4r::tas_partition_local_dfs",
27                     iptr_of!(graph),
28                     partition_ptr,
29                     v,
30                     iptr_of!(seach_state),
31                 );
32             }
33             return;
34         }
35         state.mark_discovered(v);
36         for w in partition.adjacencies.get(&v).unwrap() {
37             state.stack.push(*w);
38         }
39     }
40 }

```

Fig. 17. Partition Search

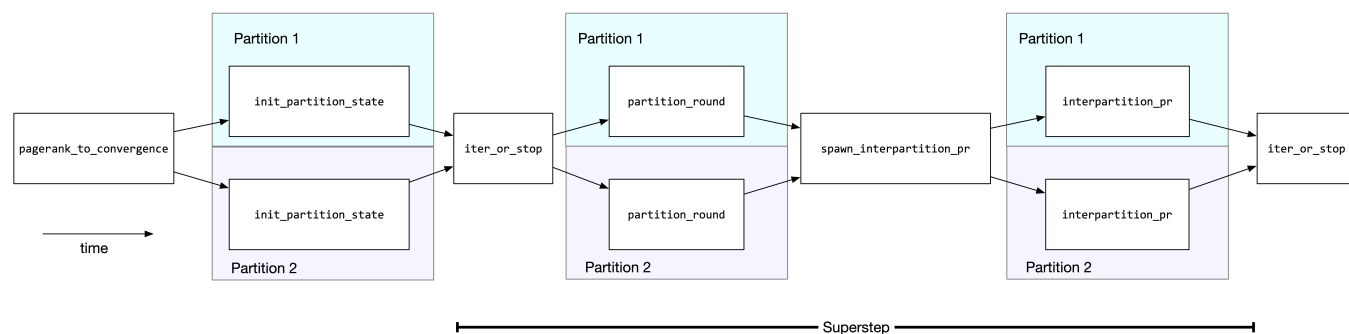


Fig. 18. PageRank Physical Plan: Example for graph with two partitions.